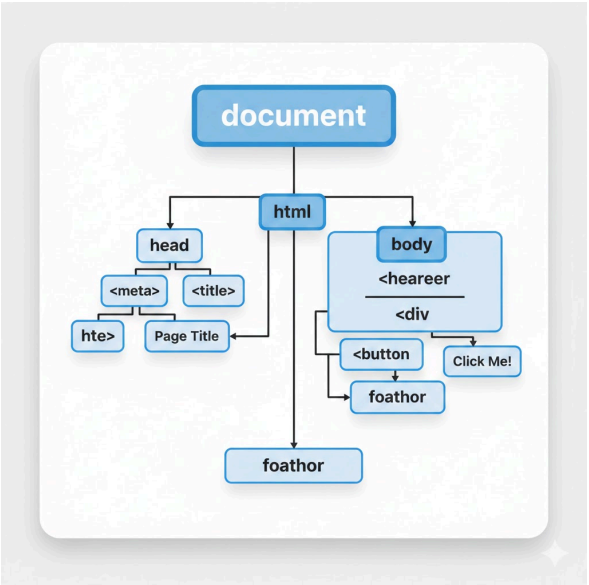
Guia de Estudos: JavaScript e Manipulação de DOM

Este documento apresenta os conceitos fundamentais do Modelo de Objeto de Documento (DOM), métodos de seleção de elementos, manipulação de eventos e estruturas de controle em JavaScript.

1. O que é a Árvore DOM?

O **DOM (Document Object Model)** é uma interface de programação que representa documentos HTML e XML como uma estrutura de árvore. Cada ramificação da árvore termina em um nó, e cada nó contém objetos que podem ser manipulados via JavaScript.



2. Seleção de Elementos (Tags)

Para interagir com o HTML, o JavaScript oferece diferentes métodos para localizar e selecionar elementos na página.

Seleção por Tag (Marca)

Utilizada quando queremos selecionar elementos pelo nome da tag (ex: `p`, `h1`, `div`).

- **Método:** `getElementsByTagName()`
- **Nota:** Caso exista mais de um elemento com a mesma tag, o retorno será uma coleção. Para acessar um item específico, utiliza-se o índice em colchetes `[ ]`.
    - *Exemplo:* `document.getElementsByTagName('p')[0]` (seleciona o primeiro parágrafo).

Seleção por ID

É a forma mais eficiente de selecionar um elemento único.

- **Método:** `getElementById('id_do_elemento')`

Seleção por Nome (Name)

Muito comum em formulários.

- **Método:** `getElementsByName()`
- **Nota:** Como retorna uma lista (NodeList), é necessário utilizar o índice, geralmente `[0]`, para selecionar o primeiro elemento encontrado.

Seleção por Classe (Class)

Seleciona elementos baseando-se no atributo `class`.

- **Método:** `getElementsByClassName('nome_da_classe')`

Seleção por Seletores (Query Selector)

Uma forma moderna e versátil que utiliza a mesma sintaxe dos seletores CSS.

- **Para Tags/DIVs:** `document.querySelector('div')`
- **Para Classes:** `document.querySelector('.nome-da-classe')`
- **Para IDs:** `document.querySelector('#id-do-elemento')`

3. Eventos de DOM

Eventos são ações que ocorrem no sistema que você está programando — o sistema informa sobre essas ações para que você possa responder a elas.

- **Evento `onclick` via HTML:** Ocorre quando o usuário clica em um elemento. É declarado diretamente na tag HTML (ex: `<button onclick="minhaFuncao()">`).
- **Eventos via JavaScript:** É a forma recomendada para manter o código organizado (separando estrutura de comportamento).
    - *Exemplo:* `elemento.addEventListener('click', funcao);`

4. Estruturas de Controle e Condições

Condicional `if` (Se)

Utilizada para executar um bloco de código apenas se uma condição for verdadeira.

Uso Direto no JS: ```javascript if (velocidade > 60) { console.log("Você foi multado!"); }

-

Estrutura de Escolha `switch`

Ideal para testar múltiplas condições fixas para uma única variável.

- **Importante:** Cada caso deve ser finalizado com o comando `break` para interromper a execução e evitar que os casos seguintes sejam executados sem necessidade.

5. Estruturas de Repetição (Laços)

`while` (Enquanto)

O bloco de código é executado **enquanto** a condição especificada for verdadeira. A verificação ocorre antes da execução do bloco.

```
while (condicao) {
   // código a ser executado
}
```

`do while` (Faça enquanto)

Semelhante ao `while`, mas a verificação ocorre **após** a primeira execução. Isso garante que o código seja executado pelo menos uma vez, mesmo que a condição seja falsa desde o início.

6. Estrutura de Repetição com Variável de Controle (`for`)

A estrutura `for` é uma forma otimizada de realizar repetições quando se conhece previamente o limite da iteração. Ela agrupa a inicialização, o teste lógico e o incremento em uma única linha de comando, tornando o código mais limpo e legível.

+1

Sintaxe e Funcionamento

A estrutura é composta por três partes fundamentais separadas por ponto e vírgula:

```
for (inicio; teste; incr) {

   // Bloco de código a ser repetido
```

}

- **Início (Inicialização):** É executado apenas uma vez, no começo do laço. Geralmente serve para declarar e inicializar a variável de controle.
- **Teste (Condição):** Uma expressão lógica que é verificada antes de cada volta. O bloco só é executado se o resultado for verdadeiro.
- **Incr (Incremento):** Executado ao final de cada iteração, atualizando a variável de controle (ex: `c++`) antes da próxima verificação.

Representação Visual (Fluxograma)

O fluxo do `for` segue um ciclo organizado onde o teste sempre precede a execução do conteúdo:

1. **Inicialização** da variável.
2. Realização do **Teste Lógico**.
3. Se verdadeiro, executa o **Bloco de Código**.
4. Executa o **Incremento** e retorna ao passo 2.

Exemplo de Implementação

Enquanto no `while` os elementos ficam espalhados, no `for` eles são centralizados:

```
// Exemplo: Contando de 1 até 6

for (var c = 1; c <= 6; c++)

 { console.log('Passo ' + c); }
```

Utilize o `for` sempre que você tiver um número definido de iterações, como ao percorrer todos os itens de uma lista (Array) ou todos os elementos selecionados por uma classe no seu HTML

```
let valores = [8, 1, 7, 4, 2, 9];

for(let pos = 0; pos < valores.length; pos++) {

    console.log(`A posição ${pos} tem o valor ${valores[pos]}`);

}
```

Método Simplificado (`for...in`):

Exclusivo para objetos e arrays, esta sintaxe é mais legível e direta:

```
for(let pos in valores) {
```

```
    console.log(valores[pos]);
}
```

Dica de Estudo: Lembre-se que o método `indexOf(valor)` é muito útil para buscar a posição de um item específico dentro do vetor. Se ele retornar `-1`, significa que o valor não foi encontrado

7. Variáveis Compostas (Arrays / Vetores)

Um **Array** (ou Vetor) é uma variável especial capaz de armazenar diversos valores em uma estrutura única, acessíveis por meio de índices numéricos. Diferente de uma variável simples, o vetor permite organizar grandes coleções de dados sob um único nome.

Estrutura e Índices

Os dados em um vetor são organizados em "vagas" chamadas de **elementos**.

- **Índice (Key):** É a posição do valor dentro do array. Em JavaScript, a contagem sempre começa no **0**.
- **Elemento:** É a composição de um valor e o seu respectivo índice.

Operações Fundamentais

Com base no exemplo `let num = [2, 8, 6, 5, 7, 0, 9]`, as principais formas de manipular os dados são:

- **Atributo `.length`:** Informa a quantidade total de elementos (comprimento) do vetor.
- **Método `.push(valor)`:** Adiciona um novo valor automaticamente na última posição, sem necessidade de informar o índice.
- **Método `.sort()`:** Utilizado para ordenar os elementos.
    - **Atenção:** Por padrão, ele ordena como texto (strings), o que pode causar erros em números (ex: 25 vindo antes de 6).
    - **Correção Numérica:** Para ordem crescente correta, utiliza-se: `num.sort((a, b) => a - b);`.
- **Acesso Direto:** Para exibir um item específico, usa-se o nome da variável seguido do índice entre colchetes, como `num[0]`.

Percorrendo o Vetor

Para manipular ou exibir todos os itens de um array de forma eficiente, utilizamos as estruturas de repetição:

Método Tradicional (`for`):

Ideal quando se precisa de controle total sobre o índice.

JavaScript

```
for(let pos = 0; pos < num.length; pos++) {
    console.log(`A posição ${pos} tem o valor ${num[pos]}`);
}
```

Método Simplificado (`for...in`):

Uma sintaxe mais moderna e direta para percorrer arrays e objetos:

JavaScript

```
for(let pos in num) {
    console.log(num[pos]);
}
```

Exemplo Consolidado

JavaScript

```
let num = [2, 8, 6, 5, 7, 0, 9];

num.push(1);              // Adiciona 1 ao final
num.sort((a, b) => a - b); // Ordena: [0, 1, 2, 5, 6, 7, 8, 9]

console.log(`O vetor tem ${num.length} posições`);
console.log(`O primeiro valor é ${num[0]}`);
```

Dica de Estudo: Use o método `num.indexOf(valor)` para encontrar em qual índice um número específico está guardado. Se o valor não existir no array, o JavaScript retornará `-1`.

Exemplo percorrendo array com for:

```
for(let pos = 0; pos < valores.length ; pos++){

    console.log(`A posição ${pos} tem o valor ${valores[pos]}`)

}
```

For do for para arrays e objetos (exclusivo para percorrer arrays (variável composta) e objetos)

```
for(let pos in valores){
    console.log(`A posição ${pos} tem o valor ${valores[pos]}`)

}
```

8. Funções em JavaScript

Uma **Função** é um bloco de código que realiza uma tarefa específica e só é executado quando é "chamado" ou "invocado". Elas são fundamentais para poupar repetições e organizar a lógica do programa.

Estrutura de uma Função

Toda função é composta por elementos principais que definem como ela recebe dados e o que ela devolve para o sistema:

- **Chamada:** É o comando que dá início à execução da função (ex: `acao(parametro)`).
- **Parâmetros:** São valores que a função recebe para trabalhar. Uma função pode ter vários parâmetros ou nenhum.
- **Ação:** É o conjunto de instruções (o código) que será executado dentro das chaves `{ }`.
- **Retorno:** É o resultado final que a função devolve para quem a chamou. Nem toda função possui um retorno visual, mas todas processam algo.

Funções com Condições

Podemos unir funções com as estruturas que você já estudou, como o `if/else`, para criar códigos inteligentes.

Exemplo: Verificador de Par ou Ímpar

```
function parimpar(n) {
    if (n % 2 == 0) {
        return 'Par!'
    } else {
        return 'Ímpar!'
    }
}


let res = parimpar(11) // Chamada da função passando o valor 11
console.log(res)      // Resultado: Ímpar!
```

Funções no DOM (Eventos)

No seu estudo de JavaScript para a web, você usará muito as funções para responder a cliques de usuários:

+1

- No HTML: Você vincula uma função a um evento como o `onclick`.
  +1
- No JavaScript: Você usa o `addEventListener` para "escutar" o clique e disparar a função correspondente.
  +1

Diferença entre Parâmetros e Retorno

- Parâmetro: É o que entra na função para ser processado.
- Retorno: É o que sai da função após o processamento.

Dica de Estudo: Pense na função como uma "fábrica": você entrega a matéria-prima (parâmetros), ela faz o trabalho (ação) e te entrega o produto final (retorno)**.**

9. Objetos em JavaScript (`Object`)

Um **Objeto** é uma variável composta que agrupa características (propriedades) e comportamentos (métodos) de um elemento em uma única estrutura. É a base para

organizar dados complexos que uma variável simples ou um array não conseguiriam descrever com clareza.

Estrutura de um Objeto

Diferente dos arrays que usam colchetes `[ ]`, os objetos utilizam chaves `{ }` e funcionam com o sistema de **Par: Chave e Valor**.

- **Propriedades:** São as características do objeto (ex: nome, idade, peso).
- **Métodos:** São funções guardadas dentro do objeto que descrevem o que ele pode fazer (ex: engordar).

Exemplo Prático: Criando e Manipulando um Objeto

Com base no seu exemplo de código:

```
let amigo = {

    nome: 'Maria',

    sexo: 'F',

    Idade: 22,

    peso: 65.5,

    engordar(p = 0) {

        console.log('Engordou')

        this.peso += p // O comando "this" refere-se ao próprio objeto amigo

    }

}



amigo.engordar(2) // Chamada do método passando o valor 2

console.log(`${amigo.nome} pesa ${amigo.peso} kg`) // Resultado: Maria pesa 67.5 kg
```

Principais Diferenças: Array vs. Objeto

- **Array:** Ideal para listas de itens semelhantes (ex: lista de alunos, lista de compras).
- **Objeto:** Ideal para descrever um item único com várias qualidades diferentes (ex: um perfil de usuário, um produto de loja).

Dica de Estudo: O uso do `this` dentro de um método é fundamental para acessar as outras propriedades do mesmo objeto sem precisar repetir o nome da variável principal.